

AI & Machine Learning

Complete 8-Week Study Notes

From Python Foundations to Deployed Deep Learning & NLP Systems

Concepts • Math & Formulas • Full Code Examples • Evaluation Metrics
Real Projects • Interview Q&A • Career & Deployment Guide

Compiled by Ez Tech Explains

Companion notes to the AI & ML 8-Week Roadmap

Table of Contents

■ Prerequisites & How to Use This Guide

■ Week 1 — Python Foundations & Mathematics

NumPy, Pandas, Matplotlib, Linear Algebra, Statistics, Calculus

■ Week 2 — Data Preprocessing & EDA

Missing values, outliers, encoding, scaling, correlation

■ Week 3 — Regression Models

Linear, Ridge, Lasso, Gradient Descent, MAE/MSE/RMSE/R2

■ Week 4 — Classification Models

Logistic Regression, KNN, Naive Bayes, Confusion Matrix, ROC-AUC

■ Week 5 — Tree-Based Models & Ensembles

Decision Trees, Random Forest, AdaBoost, GBM, XGBoost

■ Week 6 — Unsupervised Learning & Dim. Reduction

K-Means, Hierarchical, DBSCAN, PCA, t-SNE

■ Week 7 — Deep Learning & Computer Vision

ANN, Backpropagation, CNNs, Transfer Learning

■ Week 8 — NLP, Transformers & Deployment

Tokenization, Embeddings, Attention, BERT/GPT, FastAPI, Docker

■ Career Readiness — Portfolio, Resume & Interview Prep

70-20-10 rule, bias-variance, interview rounds, checklist

How to Use This Guide

This is a companion deep-dive to your 8-Week AI/ML Roadmap PDF. Each week section includes: the core concepts explained in plain language, the underlying math/formulas, runnable Python code you can copy into Colab/Jupyter, evaluation metrics tables, the week's project brief, and an interview quick-fire Q&A; block. Prerequisites: basic computer knowledge, no prior ML required, 2-4 hours/day commitment.

Recommended workflow per week: (1) read the concept + math, (2) run every code block yourself in Google Colab, (3) rebuild the project from scratch without copy-pasting, (4) answer the interview Q&A; out loud from memory before moving on.

Python Foundations & Mathematics

NumPy • Pandas • Matplotlib • Linear Algebra • Statistics • Calculus

1.1 Why This Week Matters

Every ML model is just math implemented in code, applied to data held in arrays and dataframes. Before touching any algorithm, you must be fluent in manipulating data (NumPy/Pandas), visualizing it (Matplotlib/Seaborn), and understanding the mathematical objects models are built from (vectors, matrices, probability, derivatives).

1.2 NumPy — Numerical Computing

NumPy arrays (**ndarray**) are the backbone of all ML libraries. They are faster than Python lists because they are stored in contiguous memory and operations are vectorized (run in compiled C code, not a Python loop).

```
import numpy as np

# Creating arrays
a = np.array([1, 2, 3, 4])
zeros = np.zeros((3, 3))
ones = np.ones((2, 4))
rng = np.arange(0, 10, 2)      # [0 2 4 6 8]
lin = np.linspace(0, 1, 5)    # 5 evenly spaced points

# Shape & reshape
m = np.arange(12).reshape(3, 4)
print(m.shape)                # (3, 4)
print(m.T)                    # transpose

# Vectorized operations (no for-loops!)
x = np.array([1, 2, 3])
y = np.array([4, 5, 6])
print(x + y, x * y, x.dot(y)) # element-wise + dot product

# Broadcasting: operate on arrays of different shapes
mat = np.ones((3, 3))
print(mat + np.array([1, 2, 3])) # adds row-vector to every row

# Aggregations
print(m.sum(), m.mean(axis=0), m.std(), m.max(axis=1))

# Boolean indexing / filtering
arr = np.array([5, -2, 9, -7, 3])
print(arr[arr > 0])           # [5 9 3]

# Random numbers (reproducible)
np.random.seed(42)
noise = np.random.randn(5)    # standard normal samples
```

1.3 Pandas — Data Manipulation

Pandas gives you the **DataFrame**, a labeled table (rows + named columns), the standard structure for tabular data in every data science pipeline.

```
import pandas as pd
```

```

df = pd.read_csv("netflix_titles.csv")
df.head()           # first 5 rows
df.info()           # dtypes + non-null counts
df.describe()       # mean, std, quartiles for numeric cols
df.shape            # (rows, cols)

# Selecting data
df["type"]           # single column -> Series
df[["title", "release_year"]] # multiple columns -> DataFrame
df.loc[0:5, "title"] # label-based indexing
df.iloc[0:5, 0:3]    # position-based indexing

# Filtering
movies = df[df["type"] == "Movie"]
recent = df[(df["release_year"] >= 2020) & (df["type"] == "Movie")]

# Handling missing data
df.isnull().sum()    # count NaNs per column
df.dropna(subset=["director"]) # drop rows with missing director
df["country"].fillna("Unknown", inplace=True)

# Grouping & aggregation
df.groupby("type")["release_year"].mean()
df.groupby("country").size().sort_values(ascending=False).head(10)

# Creating new columns
df["decade"] = (df["release_year"] // 10) * 10

# Sorting & merging
df.sort_values("release_year", ascending=False)
pd.merge(df1, df2, on="id", how="left")

# Apply / lambda
df["title_len"] = df["title"].apply(lambda x: len(x))

```

1.4 Matplotlib & Seaborn — Visualization

```

import matplotlib.pyplot as plt
import seaborn as sns

# Line plot
plt.plot(df.groupby("release_year").size())
plt.title("Titles Released per Year"); plt.xlabel("Year"); plt.ylabel("Count")
plt.show()

# Bar chart
sns.countplot(data=df, x="type")

# Histogram / distribution
sns.histplot(df["release_year"], bins=20, kde=True)

# Correlation heatmap
sns.heatmap(df.corr(numeric_only=True), annot=True, cmap="coolwarm")

# Boxplot for outliers
sns.boxplot(x=df["duration_minutes"])

# Scatter plot for relationships
sns.scatterplot(data=df, x="budget", y="revenue", hue="genre")

```

1.5 Mathematics for Machine Learning

A) Linear Algebra Basics

Data is represented as vectors and matrices. A dataset with n rows and m features is a matrix of shape (n, m) . Model weights are vectors. Understanding these operations explains how predictions are computed.

Linear regression prediction as a dot product:

$$\hat{y} = \mathbf{w}^T \mathbf{x} + b = w_1x_1 + w_2x_2 + \dots + w_nx_n + b$$

- **Vector:** ordered list of numbers, e.g. a feature row [age, income, score].
- **Matrix:** a 2D grid of numbers — your whole dataset (rows = samples, columns = features).
- **Dot product:** sum of element-wise multiplication — the core of every linear model and neural network layer.
- **Matrix multiplication:** stacking dot products — how a neural network layer transforms an input in one shot.
- **Identity, Transpose, Inverse:** used in the closed-form Normal Equation for linear regression: $\mathbf{w} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$
- **Eigenvalues/Eigenvectors:** the mathematical basis of PCA (Week 6) — they reveal directions of maximum variance in data.

B) Statistics Fundamentals

- **Mean, Median, Mode:** measures of central tendency.
- **Variance & Standard Deviation:** spread of data around the mean — critical for scaling features.
- **Probability distributions:** Normal (Gaussian), Binomial, Poisson — assumptions many models rely on.
- **Correlation vs Causation:** correlation (Pearson's r , range -1 to 1) measures linear relationship strength; it never proves one variable causes another.
- **Hypothesis testing & p-value:** used to validate whether an observed effect (e.g. A/B test lift) is statistically significant (commonly $p < 0.05$).
- **Bayes' Theorem:** $P(A|B) = P(B|A) \cdot P(A) / P(B)$ — foundation of the Naive Bayes classifier (Week 4).

Standard deviation formula:

$$\sigma = \sqrt{(\sum (x_i - \mu)^2) / N}$$

C) Calculus Basics

- **Derivative:** rate of change of a function — tells us the slope of the loss curve at a point.
- **Partial derivative:** derivative with respect to one variable while others are held constant — used when a model has many weights.
- **Gradient:** vector of all partial derivatives; it points in the direction of steepest increase of the loss function.
- **Chain Rule:** lets us compute gradients through composed functions — this is backpropagation in neural networks (Week 7).

Gradient Descent update rule (used in almost every ML/DL model):

$$\mathbf{w} := \mathbf{w} - \alpha \cdot (\partial L / \partial \mathbf{w})$$

where α (alpha) is the learning rate and L is the loss function.

```
# A tiny from-scratch gradient descent for  $y = wx + b$ 
import numpy as np
X = np.array([1, 2, 3, 4, 5])
y = np.array([3, 5, 7, 9, 11]) # true relation:  $y = 2x + 1$ 

w, b = 0.0, 0.0
lr = 0.01
for epoch in range(1000):
    y_pred = w * X + b
    error = y_pred - y
    dw = (2 / len(X)) * np.sum(error * X) # dL/dw
    db = (2 / len(X)) * np.sum(error)      # dL/db
    w -= lr * dw
```

```
b -= lr * db

print(f"Learned: y = {w:.2f}x + {b:.2f}")    # ~ y = 2.00x + 1.00
```

■ **PROJECT: Netflix Data Analysis**

Load the Netflix titles dataset, clean missing values, and answer business questions using Pandas + visualize insights with Matplotlib/Seaborn.

Dataset: Netflix Movies and TV Shows dataset (Kaggle)

- Load data, inspect shape/dtypes/nulls with `df.info()` and `df.isnull().sum()`.
- Clean: fill missing 'country'/'director' with 'Unknown', drop rows with no 'date_added'.
- Analyze: titles added per year, most common genres, top content-producing countries.
- Visualize: bar chart of Movie vs TV Show count, trend line of releases over years, heatmap of correlations.
- Conclude with 3-4 written insights — this is what a recruiter/interviewer will ask about.

1.6 Interview Quick-Fire (Week 1)

Q: Why is NumPy faster than native Python lists?

A: NumPy arrays store elements of a single fixed type in contiguous memory blocks and use vectorized C-level operations (SIMD), avoiding Python's per-element loop overhead and type checks.

Q: What is the difference between `.loc` and `.iloc` in Pandas?

A: `.loc` selects by label/index name (and is inclusive of the end label), while `.iloc` selects by integer position (exclusive of the end index), similar to Python list slicing.

Q: What is broadcasting in NumPy?

A: It's the mechanism that lets NumPy perform element-wise operations on arrays of different but compatible shapes by virtually 'stretching' the smaller array without copying data.

Q: Why do we need the learning rate in gradient descent, and what happens if it's too high/low?

A: It controls the step size when updating weights. Too high -> the loss can diverge/oscillate and overshoot the minimum. Too low -> training becomes extremely slow and can get stuck.

Data Preprocessing & Exploratory Data Analysis

Missing Values • Outliers • Encoding • Scaling • Correlation

2.1 Why Preprocessing Is 80% of Real ML Work

Real-world data is messy: missing entries, inconsistent formats, outliers, and categorical text that models can't read directly. A model is only as good as the data fed into it — "garbage in, garbage out". Interviewers care heavily about this stage.

2.2 Handling Missing Values

Strategy	When to Use	Code
Drop rows/columns	Very few missing rows OR column mostly empty (>60%)	<code>df.dropna()</code>
Mean/Median imputation	Numeric column, no severe outliers (mean) or skewed (median)	<code>df[col].fillna(df[col].median())</code>
Mode imputation	Categorical columns	<code>df[col].fillna(df[col].mode()[0])</code>
Forward/Backward fill	Time-series data	<code>df[col].fillna(method='ffill')</code>
Model-based (KNN/Iterative)	Large datasets, missing not random	from sklearn.impute import KNNImputer

```
from sklearn.impute import SimpleImputer, KNNImputer

num_imputer = SimpleImputer(strategy="median")
df[["age", "income"]] = num_imputer.fit_transform(df[["age", "income"]])

cat_imputer = SimpleImputer(strategy="most_frequent")
df[["city"]] = cat_imputer.fit_transform(df[["city"]])

# KNN-based imputation (uses similar rows to estimate missing value)
knn_imp = KNNImputer(n_neighbors=5)
df_num = knn_imp.fit_transform(df.select_dtypes(include="number"))
```

2.3 Outlier Detection

Outliers can distort means, variances, and model coefficients (especially in linear models). Two classic detection methods:

IQR method: outlier if $x < Q1 - 1.5 \cdot IQR$ or $x > Q3 + 1.5 \cdot IQR$

Z-score method: outlier if $|z| > 3$, where $z = (x - \mu) / \sigma$

```
Q1 = df["price"].quantile(0.25)
Q3 = df["price"].quantile(0.75)
IQR = Q3 - Q1
lower, upper = Q1 - 1.5 * IQR, Q3 + 1.5 * IQR
df_clean = df[(df["price"] >= lower) & (df["price"] <= upper)]

from scipy import stats
z_scores = stats.zscore(df["price"])
```



```
df_clean = df[(abs(z_scores) < 3)]
```

2.4 Feature Encoding (Text → Numbers)

Encoding	Best For	Notes
Label Encoding	Ordinal categories (Low/Medium/High)	Assigns 0,1,2... — implies order
One-Hot Encoding	Nominal categories (city, color)	Creates a binary column per category; avoid for high-card
Target/Mean Encoding	High-cardinality categoricals	Replace category with mean of target; risk of leakage — u
Frequency Encoding	High-cardinality, tree models	Replace category with its frequency count

```
from sklearn.preprocessing import LabelEncoder, OneHotEncoder
import pandas as pd

le = LabelEncoder()
df["size_encoded"] = le.fit_transform(df["size"]) # Low=0, Medium=1, High=2 (alphabetical by default)

df_onehot = pd.get_dummies(df, columns=["city"], drop_first=True) # avoids dummy variable trap

ohe = OneHotEncoder(sparse_output=False, handle_unknown="ignore")
encoded = ohe.fit_transform(df[["color"]])
```

2.5 Feature Scaling

Algorithms based on distance (KNN, K-Means, SVM) or gradient descent (linear/logistic regression, neural nets) need features on comparable scales; tree-based models (Decision Tree, Random Forest, XGBoost) do NOT require scaling.

Standardization (Z-score): $x' = (x - \mu) / \sigma \rightarrow \text{mean } 0, \text{ std } 1$

Normalization (Min-Max): $x' = (x - \min) / (\max - \min) \rightarrow \text{range } [0, 1]$

```
from sklearn.preprocessing import StandardScaler, MinMaxScaler

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test) # NEVER fit on test data (data leakage!)

mm = MinMaxScaler()
X_norm = mm.fit_transform(X_train)
```

■ Golden rule: always `fit_transform()` on training data only, then `transform()` the test/validation data with the same fitted scaler. Fitting on the full dataset before splitting is a common cause of data leakage.

2.6 Correlation Analysis

```
corr = df.corr(numeric_only=True)
sns.heatmap(corr, annot=True, cmap="coolwarm", fmt=".2f")

# Drop one of a highly correlated pair (multicollinearity) to keep linear models stable
high_corr = corr[(corr.abs() > 0.85) & (corr != 1.0)]
```

Multicollinearity (features highly correlated with each other) inflates the variance of linear regression coefficients, making them unstable and hard to interpret. Detect it with correlation heatmaps or VIF (Variance Inflation Factor).

■ **PROJECT: Customer Churn Analysis**

Take a raw telecom customer dataset and transform it into a model-ready dataset through a full EDA + cleaning pipeline.

Dataset: Telco Customer Churn dataset (Kaggle/IBM)

- Check dtypes — TotalCharges often loads as text; convert with `pd.to_numeric(errors='coerce')`.
- Impute missing values with median/mode strategy.
- Encode categorical columns (Contract, PaymentMethod) via one-hot encoding.
- Scale numeric columns (tenure, MonthlyCharges) with StandardScaler.
- Plot churn rate by contract type and tenure buckets — summarize key churn drivers.

2.7 Interview Quick-Fire (Week 2)

Q: Why should you fit a scaler only on training data?

A: Fitting on the full dataset lets information about the test set (its mean/variance) leak into training, giving an overly optimistic evaluation that won't hold on truly unseen data.

Q: When would you use one-hot encoding vs label encoding?

A: Label encoding is appropriate for ordinal features where order is meaningful; one-hot encoding is safer for nominal features because label encoding would falsely imply a ranking between categories.

Q: How do you decide between mean and median imputation?

A: Use mean when the feature is roughly symmetric with no major outliers; use median when the distribution is skewed or contains outliers, since the median is robust to extreme values.

Regression Models

Linear • Ridge • Lasso • Gradient Descent • Evaluation Metrics

3.1 Linear Regression

Linear Regression models the relationship between input features and a continuous target as a weighted sum.

$$\hat{y} = w_1x_1 + w_2x_2 + \dots + w_nx_n + b$$

Cost Function — Mean Squared Error (MSE), the function training tries to minimize:

$$J(w) = (1/n) \sum (y_i - \hat{y}_i)^2$$

```
from sklearn.linear_model import LinearRegression
from sklearn.model_selection import train_test_split
from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
import numpy as np

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

model = LinearRegression()
model.fit(X_train, y_train)
y_pred = model.predict(X_test)

print("Coefficients:", model.coef_, "Intercept:", model.intercept_)
print("MAE:", mean_absolute_error(y_test, y_pred))
print("MSE:", mean_squared_error(y_test, y_pred))
print("RMSE:", np.sqrt(mean_squared_error(y_test, y_pred)))
print("R2:", r2_score(y_test, y_pred))
```

3.2 Gradient Descent — How Models Actually Learn

Instead of solving for weights directly (which is only feasible for small datasets via the Normal Equation), most ML training iteratively nudges weights in the direction that reduces the loss.

- **Batch Gradient Descent:** uses the entire dataset per update — accurate but slow on big data.
- **Stochastic Gradient Descent (SGD):** uses one sample per update — fast, noisy convergence.
- **Mini-batch Gradient Descent:** uses small batches (e.g. 32/64 samples) — the practical default, used by almost every deep learning framework.

3.3 Regularization — Ridge & Lasso Regression

Regularization adds a penalty term to the loss function to discourage overly large weights, which reduces overfitting.

$$\text{Ridge (L2): } J(w) = \text{MSE} + \lambda \sum w_i^2$$

$$\text{Lasso (L1): } J(w) = \text{MSE} + \lambda \sum |w_i|$$

- **Ridge** shrinks coefficients toward zero but rarely makes them exactly zero — good when all features are somewhat useful.
- **Lasso** can shrink coefficients to exactly zero — performs automatic feature selection.
- **Elastic Net** combines both penalties, useful when features are correlated.

- λ (alpha in sklearn) controls regularization strength — tune it via cross-validation (GridSearchCV).

```
from sklearn.linear_model import Ridge, Lasso, ElasticNet
from sklearn.model_selection import GridSearchCV

ridge = Ridge(alpha=1.0).fit(X_train, y_train)
lasso = Lasso(alpha=0.1).fit(X_train, y_train)

# Finding the best alpha automatically
params = {"alpha": [0.001, 0.01, 0.1, 1, 10, 100]}
grid = GridSearchCV(Ridge(), params, cv=5, scoring="neg_mean_squared_error")
grid.fit(X_train, y_train)
print(grid.best_params_)
```

3.4 Evaluation Metrics for Regression

Metric	Formula	Interpretation
MAE	$\text{avg}(y - \hat{y})$	Average absolute error, same unit as target, robust to outliers
MSE	$\text{avg}((y - \hat{y})^2)$	Penalizes large errors more; not in original unit
RMSE	$\sqrt{\text{MSE}}$	Back in original unit; still penalizes big errors heavily
R ² Score	$1 - \text{SS}_{\text{res}}/\text{SS}_{\text{tot}}$	Fraction of variance explained; 1.0 = perfect, 0 = as good as mean baseline

3.5 Assumptions of Linear Regression (common interview topic)

- Linearity: relationship between X and y is linear.
- Independence of errors (no autocorrelation).
- Homoscedasticity: constant variance of residuals across all predicted values.
- Normality of residuals.
- No (or low) multicollinearity between features.

■ PROJECT: House Price Prediction

Predict house prices from features like area, bedrooms, location, and age using regression models.

Dataset: Boston Housing / Kaggle House Prices dataset

- EDA: check distribution of target 'price' (often right-skewed — consider log-transform).
- Preprocess: encode categorical features, scale numeric ones.
- Train Linear, Ridge, and Lasso regression; compare RMSE and R² across all three.
- Plot predicted vs actual values and the residual plot to check assumptions.
- Explain in plain English which features most affect price (from coefficients).

3.6 Interview Quick-Fire (Week 3)

Q: Why use RMSE over MSE for reporting model performance?

A: RMSE is expressed in the same unit as the target variable, so it's directly interpretable, while MSE is in squared units and harder to explain to stakeholders.

Q: What's the difference between Ridge and Lasso regression?

A: Ridge uses an L2 penalty that shrinks coefficients toward zero without eliminating them, while Lasso uses an L1 penalty that can shrink some coefficients to exactly zero, effectively performing feature selection.

Q: What does an R² of 0.85 mean?

A: It means the model explains 85% of the variance in the target variable using the input features; the remaining 15% is unexplained by the model.

Q: What happens if the learning rate in gradient descent is too large?

A: The updates overshoot the minimum of the loss function, and training can oscillate or diverge instead of converging.

Classification Models

Logistic Regression • KNN • Naive Bayes • Evaluation Metrics

4.1 Logistic Regression

Despite the name, this is a classification algorithm. It applies the sigmoid function to a linear combination of features to output a probability between 0 and 1.

$$\text{Sigmoid: } \sigma(z) = 1 / (1 + e^{-z}), \text{ where } z = w^T x + b$$

Loss function — Binary Cross-Entropy (Log Loss):

$$J = -(1/n) \sum [y_i \cdot \log(\hat{y}_i) + (1-y_i) \cdot \log(1-\hat{y}_i)]$$

```
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import classification_report, confusion_matrix, roc_auc_score

clf = LogisticRegression(max_iter=1000)
clf.fit(X_train, y_train)
y_pred = clf.predict(X_test)
y_proba = clf.predict_proba(X_test)[:, 1]    # probability of class 1

print(classification_report(y_test, y_pred))
print("ROC-AUC:", roc_auc_score(y_test, y_proba))
```

4.2 K-Nearest Neighbors (KNN)

A lazy, instance-based learner: to classify a new point, it looks at the K closest training points (by distance) and takes a majority vote.

$$\text{Euclidean Distance: } d(p,q) = \sqrt{(\sum (p_i - q_i)^2)}$$

- Small K → sensitive to noise (overfitting / high variance).
- Large K → smoother decision boundary (underfitting / high bias).
- Always scale features first — KNN is purely distance-based.
- Choose K via cross-validation; commonly an odd number to avoid ties in binary classification.

```
from sklearn.neighbors import KNeighborsClassifier
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_train_s = scaler.fit_transform(X_train)
X_test_s = scaler.transform(X_test)

knn = KNeighborsClassifier(n_neighbors=5)
knn.fit(X_train_s, y_train)
print(knn.score(X_test_s, y_test))
```

4.3 Naive Bayes

Based on Bayes' Theorem, with the "naive" assumption that all features are conditionally independent given the class. Extremely fast and works surprisingly well for text classification (spam detection, sentiment analysis).

$$P(\text{Class} | \text{Features}) \propto P(\text{Class}) \cdot \prod P(\text{Feature}_i | \text{Class})$$

```

from sklearn.naive_bayes import MultinomialNB, GaussianNB
from sklearn.feature_extraction.text import CountVectorizer

# Text example (Spam classifier - see Week 4 project)
vectorizer = CountVectorizer(stop_words="english")
X_train_vec = vectorizer.fit_transform(train_texts)
X_test_vec = vectorizer.transform(test_texts)

nb = MultinomialNB()
nb.fit(X_train_vec, y_train)
preds = nb.predict(X_test_vec)

```

4.4 Evaluation Metrics for Classification

The Confusion Matrix is the foundation of every classification metric:

	Predicted Positive	Predicted Negative
Actual Positive	True Positive (TP)	False Negative (FN)
Actual Negative	False Positive (FP)	True Negative (TN)

Metric	Formula	When It Matters Most
Accuracy	$(TP+TN)/(TP+TN+FP+FN)$	Balanced classes only — misleading on imbalanced data
Precision	$TP/(TP+FP)$	Cost of false positives is high (e.g. spam filters)
Recall (Sensitivity)	$TP/(TP+FN)$	Cost of false negatives is high (e.g. cancer/fraud detection)
F1 Score	$2 \cdot (P \cdot R) / (P + R)$	Balance between precision & recall, good for imbalanced data
ROC-AUC	Area under ROC curve	Overall ranking quality across all thresholds

4.5 Handling Imbalanced Data (very common interview topic)

- **Resampling:** oversample minority class (SMOTE) or undersample majority class.
- **Class weights:** `class_weight='balanced'` in sklearn penalizes misclassifying minority class more.
- **Better metrics:** use F1, Precision-Recall AUC instead of plain accuracy.
- **Threshold tuning:** adjust the decision threshold instead of the default 0.5.

```

from imblearn.over_sampling import SMOTE
sm = SMOTE(random_state=42)
X_res, y_res = sm.fit_resample(X_train, y_train)

```

■ PROJECT: Spam Email Classifier

Build a text classification pipeline that flags emails/SMS as spam or ham (not spam).

Dataset: SMS Spam Collection Dataset (UCI/Kaggle)

- Clean text: lowercase, remove punctuation/stopwords, optionally stem/lemmatize.
- Vectorize with CountVectorizer or TF-IDF (see Week 8 for TF-IDF theory).
- Train Multinomial Naive Bayes and Logistic Regression; compare F1 scores.
- Plot the confusion matrix; discuss the precision/recall tradeoff for spam filtering.
- Wrap the trained model in a simple `predict_spam(text)` function.

4.6 Interview Quick-Fire (Week 4)

Q: Why is accuracy a bad metric for imbalanced datasets?

A: If 95% of samples belong to one class, a model that always predicts that class gets 95% accuracy while being completely useless — precision, recall, and F1 reveal this failure while accuracy hides it.

Q: Precision vs Recall — when do you prioritize each?

A: Prioritize precision when false positives are costly (e.g., flagging a legitimate email as spam); prioritize recall when false negatives are costly (e.g., missing a fraudulent transaction or a disease case).

Q: Why does Naive Bayes work well for text despite its 'naive' independence assumption?

A: Even though word occurrences aren't truly independent, the model only needs to rank classes correctly, not estimate exact probabilities, so the assumption's bias often doesn't hurt classification accuracy much.

Q: What is the ROC curve plotting?

A: It plots the True Positive Rate against the False Positive Rate at every possible classification threshold, summarizing model performance across all thresholds via the AUC value.

Tree-Based Models & Ensemble Learning

Decision Trees • Random Forest • AdaBoost • Gradient Boosting • XGBoost

5.1 Decision Trees

A tree splits data recursively on feature thresholds to create pure (single-class or low-variance) leaf nodes. It's the building block for almost every top-performing tabular-data model used in industry today.

Entropy (impurity measure): $H(S) = -\sum p_i \cdot \log_2(p_i)$

Gini Index (impurity measure, default in sklearn): $\text{Gini}(S) = 1 - \sum p_i^2$

Information Gain = $H(\text{parent}) - \sum (|\text{child}|/|\text{parent}|) \cdot H(\text{child})$ — the tree picks the split that maximizes this at each node.

```
from sklearn.tree import DecisionTreeClassifier, plot_tree
import matplotlib.pyplot as plt

dt = DecisionTreeClassifier(max_depth=5, criterion="gini", min_samples_leaf=10, random_state=42)
dt.fit(X_train, y_train)

plt.figure(figsize=(16, 8))
plot_tree(dt, feature_names=X.columns, class_names=["No", "Yes"], filled=True, max_depth=3)
plt.show()

# Feature importance
importances = pd.Series(dt.feature_importances_, index=X.columns).sort_values(ascending=False)
```

Overfitting in trees: an unpruned tree memorizes training data (100% train accuracy, poor test accuracy). Control it with `max_depth`, `min_samples_split`, `min_samples_leaf`, or cost-complexity pruning (`ccp_alpha`).

5.2 Random Forest — Bagging Ensemble

Random Forest builds many decision trees on bootstrapped samples of the data (bagging), and at each split only considers a random subset of features. Predictions are averaged (regression) or majority-voted (classification). This reduces variance dramatically compared to a single tree.

```
from sklearn.ensemble import RandomForestClassifier

rf = RandomForestClassifier(n_estimators=300, max_depth=10, max_features="sqrt",
                           random_state=42, n_jobs=-1)
rf.fit(X_train, y_train)
print(rf.score(X_test, y_test))

# Feature importance (very commonly asked about in interviews)
feat_imp = pd.Series(rf.feature_importances_, index=X.columns).sort_values(ascending=False)
```

5.3 Boosting: AdaBoost, Gradient Boosting, XGBoost

Unlike bagging (parallel, independent trees), boosting builds trees sequentially — each new tree focuses on correcting the errors of the previous ones.

- **AdaBoost:** increases the weight of misclassified samples so the next weak learner focuses on them; final prediction is a weighted vote of all learners.

- **Gradient Boosting:** each new tree is trained to predict the residual errors (negative gradient of the loss) of the current ensemble.
- **XGBoost / LightGBM / CatBoost:** optimized, regularized implementations of gradient boosting — the most commonly used models in Kaggle competitions and real-world tabular ML jobs.

```
from sklearn.ensemble import AdaBoostClassifier, GradientBoostingClassifier
import xgboost as xgb

ada = AdaBoostClassifier(n_estimators=100, learning_rate=0.5, random_state=42)
ada.fit(X_train, y_train)

gb = GradientBoostingClassifier(n_estimators=200, learning_rate=0.1, max_depth=3, random_state=42)
gb.fit(X_train, y_train)

xgb_clf = xgb.XGBClassifier(
    n_estimators=300, learning_rate=0.05, max_depth=6,
    subsample=0.8, colsample_bytree=0.8, eval_metric="logloss", random_state=42
)
xgb_clf.fit(X_train, y_train,
            eval_set=[(X_test, y_test)], verbose=False)
print(xgb_clf.score(X_test, y_test))
```

5.4 Bagging vs Boosting

Aspect	Bagging (Random Forest)	Boosting (XGBoost/GBM)
Tree building	Parallel, independent trees	Sequential, each tree fixes previous errors
Goal	Reduce variance (overfitting)	Reduce bias (underfitting), then variance too
Speed	Faster to train (parallelizable)	Slower (sequential), but often more accurate
Sensitivity to noise	More robust to noisy data/outliers	Can overfit noise if not tuned

5.5 Cross-Validation & Hyperparameter Tuning

```
from sklearn.model_selection import cross_val_score, GridSearchCV, RandomizedSearchCV

scores = cross_val_score(rf, X, y, cv=5, scoring="accuracy")
print("CV Mean Accuracy:", scores.mean())

param_grid = {
    "n_estimators": [100, 200, 300],
    "max_depth": [5, 10, 15, None],
    "min_samples_split": [2, 5, 10]
}
grid = GridSearchCV(RandomForestClassifier(random_state=42), param_grid, cv=5,
                    scoring="f1", n_jobs=-1)
grid.fit(X_train, y_train)
print(grid.best_params_, grid.best_score_)
```

■ **PROJECT: Loan Approval Prediction**

Predict whether a loan application will be approved based on applicant financial and demographic data.

Dataset: Loan Prediction dataset (Kaggle/Analytics Vidhya)

- Handle missing values in LoanAmount, Credit_History, Self_Employed.
- Encode categorical variables, engineer a 'TotalIncome' feature (ApplicantIncome + CoapplicantIncome).
- Train Decision Tree, Random Forest, and XGBoost; compare accuracy/F1/ROC-AUC.
- Tune the best model with GridSearchCV and analyze feature importances.
- Discuss overfitting: compare train vs test accuracy for each model.

5.6 Interview Quick-Fire (Week 5)

Q: Why does Random Forest reduce overfitting compared to a single Decision Tree?

A: It averages predictions across many decorrelated trees (built on bootstrapped samples with random feature subsets), which cancels out individual trees' noise and reduces variance.

Q: What's the key difference between bagging and boosting?

A: Bagging trains trees independently in parallel to reduce variance, while boosting trains trees sequentially, each one correcting the errors of the previous ensemble, primarily to reduce bias.

Q: Why don't tree-based models require feature scaling?

A: Trees split on threshold comparisons for one feature at a time, so the relative scale of different features doesn't affect how splits are chosen.

Q: What does 'max_depth' control and what happens if it's too high?

A: It limits how many times the tree can split. Too high allows the tree to memorize training data (overfitting); too low causes underfitting by not capturing enough patterns.

Unsupervised Learning & Dimensionality Reduction

K-Means • Hierarchical Clustering • DBSCAN • PCA • t-SNE

6.1 What Makes Learning 'Unsupervised'

There is no target label y — the algorithm finds structure (groups, patterns, or lower-dimensional representations) directly from the input features X .

6.2 K-Means Clustering

Partitions data into K clusters by iteratively assigning points to the nearest centroid, then recomputing centroids as the mean of assigned points, until convergence.

Objective (Within-Cluster Sum of Squares / Inertia):

$$J = \sum_k \sum_{x \in C_k} \|x - \mu_k\|^2$$

```
from sklearn.cluster import KMeans
from sklearn.metrics import silhouette_score

# Elbow method to choose K
inertias = []
for k in range(1, 11):
    km = KMeans(n_clusters=k, random_state=42, n_init=10)
    km.fit(X_scaled)
    inertias.append(km.inertia_)
plt.plot(range(1, 11), inertias, marker="o")
plt.xlabel("K"); plt.ylabel("Inertia"); plt.title("Elbow Method")

km = KMeans(n_clusters=4, random_state=42, n_init=10)
labels = km.fit_predict(X_scaled)
print("Silhouette Score:", silhouette_score(X_scaled, labels))
```

- **Elbow Method:** plot inertia vs K , pick K at the 'elbow' where improvement flattens.
- **Silhouette Score:** ranges -1 to 1; measures how well-separated clusters are — higher is better.
- K-Means assumes spherical, similarly-sized clusters and is sensitive to feature scale and outliers.

6.3 Hierarchical Clustering

Builds a tree of clusters (dendrogram) either bottom-up (agglomerative — merge closest clusters iteratively) or top-down (divisive). No need to pre-specify K ; you 'cut' the dendrogram at the desired number of clusters.

```
from scipy.cluster.hierarchy import dendrogram, linkage
from sklearn.cluster import AgglomerativeClustering

Z = linkage(X_scaled, method="ward")
dendrogram(Z)
plt.title("Dendrogram")

agg = AgglomerativeClustering(n_clusters=4, linkage="ward")
labels = agg.fit_predict(X_scaled)
```

6.4 DBSCAN — Density-Based Clustering

Groups points that are closely packed (density-based) and marks points in low-density regions as noise/outliers. Unlike K-Means, it finds arbitrarily shaped clusters and doesn't require specifying the number of clusters upfront.

```
from sklearn.cluster import DBSCAN

db = DBSCAN(eps=0.5, min_samples=5)
labels = db.fit_predict(X_scaled)
n_clusters = len(set(labels)) - (1 if -1 in labels else 0)  # -1 = noise/outlier points
```

Algorithm	Needs K upfront?	Handles Noise/Outliers?	Cluster Shape
K-Means	Yes	No	Spherical / convex
Hierarchical	No (choose via dendrogram)	No	Flexible
DBSCAN	No (uses eps, min_samples)	Yes (labels as noise)	Arbitrary shape

6.5 Dimensionality Reduction: PCA

Principal Component Analysis projects high-dimensional data onto a smaller number of new axes (principal components) that capture the maximum variance in the data. These components are the eigenvectors of the data's covariance matrix, ordered by their eigenvalues (variance explained).

```
from sklearn.decomposition import PCA

pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)
print("Variance explained:", pca.explained_variance_ratio_)

# Choosing number of components to retain ~95% variance
pca_full = PCA().fit(X_scaled)
cum_var = np.cumsum(pca_full.explained_variance_ratio_)
n_components = np.argmax(cum_var >= 0.95) + 1
```

PCA use-cases: visualization (reduce to 2D/3D), removing multicollinearity, speeding up training, and noise reduction. Always scale data before PCA — it is sensitive to feature variance scale.

6.6 t-SNE

t-Distributed Stochastic Neighbor Embedding is a non-linear technique used purely for visualization (2D/3D). It preserves local neighborhood structure (similar points stay close) but distorts global distances — never use it as a preprocessing step before another model, and don't over-interpret cluster sizes/gaps.

```
from sklearn.manifold import TSNE

tsne = TSNE(n_components=2, perplexity=30, random_state=42)
X_tsne = tsne.fit_transform(X_scaled)
plt.scatter(X_tsne[:, 0], X_tsne[:, 1], c=labels, cmap="tab10")
```

■ **PROJECT: Customer Segmentation**

Segment retail customers into meaningful groups (e.g. high-value, at-risk, budget) using RFM or demographic + purchase behavior features.

Dataset: Mall Customer Segmentation dataset (Kaggle)

- Feature engineer RFM: Recency, Frequency, Monetary value per customer.
- Scale features, run the Elbow Method to pick optimal K for K-Means.
- Cluster customers, then use PCA to visualize clusters in 2D.
- Profile each cluster (e.g. 'high income, low spend' vs 'young, high spend') with business-friendly names.
- Recommend a marketing action per segment — this storytelling matters a lot in interviews.

6.7 Interview Quick-Fire (Week 6)

Q: How do you choose the number of clusters K in K-Means?

A: Common approaches are the Elbow Method (plotting inertia vs K and finding where the decrease flattens) and the Silhouette Score (choosing K that maximizes average cluster separation).

Q: What is the key difference between PCA and t-SNE?

A: PCA is a linear technique that preserves global variance structure and can be used for feature reduction before modeling; t-SNE is a non-linear technique meant only for visualization that preserves local neighborhood structure but distorts global distances.

Q: Why does K-Means struggle with non-spherical clusters, and what would you use instead?

A: K-Means assigns points based on distance to a centroid, which implicitly assumes round, similarly-sized clusters; DBSCAN or hierarchical clustering handle arbitrarily shaped clusters better.

Deep Learning & Computer Vision

Neural Networks • Backpropagation • CNNs • Transfer Learning

7.1 Artificial Neural Networks (ANN)

A neural network is a stack of layers of neurons (perceptrons). Each neuron computes a weighted sum of its inputs, adds a bias, and passes the result through a non-linear activation function.

Single neuron: $z = \sum w_{ixi} + b$, $a = \text{activation}(z)$

A) Activation Functions

Function	Formula	Used For
Sigmoid	$1/(1+e^{-z})$	Binary output layer (probability)
Tanh	$(e^z - e^{-z})/(e^z + e^{-z})$	Hidden layers (zero-centered), older RNNs
ReLU	$\max(0, z)$	Default for hidden layers — fast, avoids vanishing gradient
Softmax	$e^{z_i} / \sum e^{z_i}$	Multi-class output layer (probabilities sum to 1)

Without non-linear activations, stacking layers would collapse into a single linear function no matter how deep the network is — activations are what let neural nets learn complex, non-linear patterns.

B) Backpropagation & Training

Backpropagation uses the chain rule from calculus to compute the gradient of the loss with respect to every weight, layer by layer, from output back to input, so gradient descent can update all weights.

- **Forward pass:** compute predictions layer by layer.
- **Loss computation:** compare predictions to true labels (e.g. cross-entropy).
- **Backward pass:** propagate error gradients backward using the chain rule.
- **Weight update:** apply gradient descent (or Adam/RMSProp) to every weight.
- **Vanishing/Exploding gradients:** in deep networks, gradients can shrink to ~ 0 or blow up — mitigated by ReLU, batch normalization, residual connections, and careful weight initialization.

```
import torch
import torch.nn as nn
import torch.optim as optim

class SimpleNN(nn.Module):
    def __init__(self, input_dim):
        super().__init__()
        self.net = nn.Sequential(
            nn.Linear(input_dim, 64), nn.ReLU(),
            nn.Linear(64, 32), nn.ReLU(),
            nn.Linear(32, 1), nn.Sigmoid()
        )
    def forward(self, x):
        return self.net(x)

model = SimpleNN(input_dim=20)
criterion = nn.BCELoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)
```

```

for epoch in range(50):
    optimizer.zero_grad()
    outputs = model(X_train_tensor).squeeze()
    loss = criterion(outputs, y_train_tensor)
    loss.backward()      # backpropagation
    optimizer.step()      # weight update
    if epoch % 10 == 0:
        print(f"Epoch {epoch}, Loss: {loss.item():.4f}")

```

7.2 Convolutional Neural Networks (CNN)

CNNs are the standard architecture for images. Instead of connecting every pixel to every neuron (which doesn't scale and ignores spatial structure), they slide small learnable filters (kernels) across the image to detect local patterns like edges, textures, and shapes.

- **Convolution layer:** applies filters across the image, producing feature maps.
- **Pooling layer (Max/Average):** downsamples feature maps, reducing size and adding translation invariance.
- **Flatten + Dense layers:** at the end, feature maps are flattened and passed to fully connected layers for final classification.
- **Stride & Padding:** control how the filter moves and whether output size matches input size.

```

import tensorflow as tf
from tensorflow.keras import layers, models

model = models.Sequential([
    layers.Conv2D(32, (3,3), activation='relu', input_shape=(128,128,3)),
    layers.MaxPooling2D(2,2),
    layers.Conv2D(64, (3,3), activation='relu'),
    layers.MaxPooling2D(2,2),
    layers.Conv2D(128, (3,3), activation='relu'),
    layers.MaxPooling2D(2,2),
    layers.Flatten(),
    layers.Dense(128, activation='relu'),
    layers.Dropout(0.5),
    layers.Dense(1, activation='sigmoid')    # binary: cat vs dog
])

model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
model.summary()

from tensorflow.keras.preprocessing.image import ImageDataGenerator
train_gen = ImageDataGenerator(rescale=1./255, rotation_range=20, horizontal_flip=True,
                               zoom_range=0.2).flow_from_directory(
    "train/", target_size=(128,128), batch_size=32, class_mode="binary")
val_gen = ImageDataGenerator(rescale=1./255).flow_from_directory(
    "val/", target_size=(128,128), batch_size=32, class_mode="binary")

history = model.fit(train_gen, validation_data=val_gen, epochs=15)

```

7.3 Transfer Learning

Training a CNN from scratch needs huge datasets and compute. Transfer learning reuses a model pre-trained on a massive dataset (ImageNet) and fine-tunes it on your smaller dataset — the standard real-world approach for computer vision.

Model	Key Idea	Typical Use
VGG16	Simple, deep stack of 3x3 convolutions	Baseline, easy to understand/fine-tune
ResNet	Skip/residual connections solve vanishing gradients in very deep networks	High accuracy, widely used backbone

Model	Key Idea	Typical Use
MobileNet	Depthwise separable convolutions — lightweight	Mobile/edge deployment, faster inference

```
from tensorflow.keras.applications import MobileNetV2
from tensorflow.keras import layers, models

base_model = MobileNetV2(weights='imagenet', include_top=False, input_shape=(128,128,3))
base_model.trainable = False # freeze pretrained weights

model = models.Sequential([
    base_model,
    layers.GlobalAveragePooling2D(),
    layers.Dense(64, activation='relu'),
    layers.Dropout(0.3),
    layers.Dense(1, activation='sigmoid')
])
model.compile(optimizer='adam', loss='binary_crossentropy', metrics=['accuracy'])
model.fit(train_gen, validation_data=val_gen, epochs=10)

# Fine-tuning: unfreeze last few layers and train with a very low LR
base_model.trainable = True
for layer in base_model.layers[:-20]:
    layer.trainable = False
model.compile(optimizer=tf.keras.optimizers.Adam(1e-5), loss='binary_crossentropy', metrics=[
    'accuracy'])
model.fit(train_gen, validation_data=val_gen, epochs=5)
```

7.4 Preventing Overfitting in Deep Learning

- **Dropout:** randomly zero out neurons during training to prevent co-adaptation.
- **Data Augmentation:** random rotations/flips/zoom to artificially expand the training set.
- **Early Stopping:** stop training when validation loss stops improving.
- **Batch Normalization:** normalizes layer inputs, stabilizing and speeding up training.
- **L2 weight regularization / weight decay.**

■ *PROJECT: Cats vs Dogs Image Classifier*

Build and train a CNN (from scratch, then via transfer learning) to classify images of cats and dogs.

Dataset: Dogs vs Cats dataset (Kaggle)

- Organize data into train/val folders, apply ImageDataGenerator with augmentation.
- Train a CNN from scratch — record accuracy/loss curves for train and validation.
- Retrain using MobileNetV2 transfer learning — compare accuracy and training time.
- Plot a confusion matrix and view misclassified images to understand failure modes.
- Save the trained model (model.save('model.h5')) for deployment in Week 8.

7.5 Interview Quick-Fire (Week 7)

Q: Why do CNNs work better than plain dense networks for images?

A: CNNs use shared, local filters that exploit the spatial structure of images (nearby pixels are related), drastically reducing parameters compared to fully-connected layers and learning translation-invariant features like edges and textures.

Q: What problem does ResNet's skip connection solve?

A: It allows gradients to flow directly through shortcut paths during backpropagation, which mitigates the vanishing gradient problem and makes it possible to train much deeper networks.

Q: What is the difference between feature extraction and fine-tuning in transfer learning?

A: Feature extraction freezes the pretrained base and only trains new top layers; fine-tuning additionally unfreezes some of the pretrained layers and retrain them with a very small learning rate to adapt learned features to the new task.

Q: Why use dropout during training but not during inference?

A: Dropout randomly disables neurons to prevent overfitting and force redundancy during learning; at inference time we want the full network's capacity, so all neurons are active (with outputs appropriately scaled).

NLP, Transformers & Deployment

Tokenization • Embeddings • Attention • BERT/GPT • FastAPI • Docker

8.1 NLP Fundamentals

- **Tokenization:** splitting text into units (words/subwords/characters) a model can process.
- **Stemming:** crude chopping of word endings (e.g. 'running' → 'run') using rules — fast but can produce non-words.
- **Lemmatization:** reduces a word to its dictionary base form using vocabulary/grammar rules (e.g. 'better' → 'good') — slower but more accurate than stemming.
- **Stopwords:** common low-information words (the, is, at) often removed before classic NLP modeling (not always removed for transformers).

```
import nltk
from nltk.stem import PorterStemmer, WordNetLemmatizer
from nltk.tokenize import word_tokenize
from nltk.corpus import stopwords

text = "The runners were running faster than the running dogs"
tokens = word_tokenize(text.lower())

stemmer = PorterStemmer()
stemmed = [stemmer.stem(t) for t in tokens]          # 'runner' -> 'runner', 'running' -> 'run'

lemmatizer = WordNetLemmatizer()
lemmatized = [lemmatizer.lemmatize(t, pos='v') for t in tokens]

stop_words = set(stopwords.words('english'))
filtered = [t for t in tokens if t not in stop_words]
```

8.2 Word Embeddings — Turning Words into Vectors

Classic Bag-of-Words / TF-IDF represent text as sparse counts and ignore word meaning or order. Embeddings (Word2Vec, GloVe) map each word to a dense vector such that semantically similar words end up close together in vector space.

TF-IDF weight: $\text{tfidf}(t,d) = \text{tf}(t,d) \times \log(N / \text{df}(t))$

```
from sklearn.feature_extraction.text import TfidfVectorizer

tfidf = TfidfVectorizer(max_features=5000, ngram_range=(1,2))
X_tfidf = tfidf.fit_transform(corpus)

# Modern approach: contextual embeddings from a pretrained transformer
from sentence_transformers import SentenceTransformer
model = SentenceTransformer('all-MiniLM-L6-v2')
embeddings = model.encode(["I love machine learning", "ML is amazing"])
```

8.3 The Transformer & Attention Mechanism

Before Transformers, RNNs/LSTMs processed text sequentially, which was slow and struggled with long-range dependencies. The Transformer ("Attention Is All You Need", 2017) processes the whole sequence in parallel using self-attention, which lets every word directly attend to every other word to build context.

Self-Attention core formula:

$$\text{Attention}(Q,K,V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right) \cdot V$$

- **Query, Key, Value (Q,K,V):** learned projections of each token used to score relevance between tokens.
- **Multi-head attention:** runs several attention operations in parallel to capture different types of relationships (syntax, meaning, coreference) simultaneously.
- **Positional Encoding:** since attention has no built-in sense of order, position information is injected explicitly.

A) BERT vs GPT

Model	Architecture	Best For
BERT	Encoder-only, bidirectional (sees full context both directions)	Classification, NER, Q&A, embeddings
GPT	Decoder-only, autoregressive (predicts next token left-to-right)	Text generation, chatbots, summarization

```
from transformers import pipeline

# Sentiment analysis with a pretrained transformer (no training needed!)
sentiment = pipeline("sentiment-analysis")
print(sentiment("This product completely changed how I work!"))

# Fine-tuning BERT for text classification
from transformers import AutoTokenizer, AutoModelForSequenceClassification, Trainer,
TrainingArguments

tokenizer = AutoTokenizer.from_pretrained("bert-base-uncased")
model = AutoModelForSequenceClassification.from_pretrained("bert-base-uncased", num_labels=2)

def tokenize_fn(batch):
    return tokenizer(batch["text"], padding="max_length", truncation=True, max_length=128)

train_ds = train_ds.map(tokenize_fn, batched=True)

args = TrainingArguments(output_dir="./results", num_train_epochs=3,
                        per_device_train_batch_size=16, evaluation_strategy="epoch")
trainer = Trainer(model=model, args=args, train_dataset=train_ds, eval_dataset=val_ds)
trainer.train()
```

8.4 Large Language Models (LLMs) & Generative AI

- LLMs (GPT-4, Claude, Llama, Gemini) are massive Transformer decoders trained on huge text corpora, then aligned via instruction tuning and RLHF (Reinforcement Learning from Human Feedback).
- **Prompt Engineering:** crafting inputs (zero-shot, few-shot, chain-of-thought) to get better outputs without retraining the model.
- **RAG (Retrieval-Augmented Generation):** retrieve relevant documents from a vector database and feed them into the LLM's context to ground answers in real, up-to-date data.
- **Fine-tuning vs RAG:** fine-tuning bakes new knowledge/behavior into weights; RAG injects fresh knowledge at inference time without retraining — much cheaper and easier to keep current.

8.5 Deployment — Turning a Model into a Product

A) Serving a model with FastAPI

```
# app.py
from fastapi import FastAPI
from pydantic import BaseModel
import joblib
```

```

import numpy as np

app = FastAPI(title="ML Model API")
model = joblib.load("model.pkl")

class InputData(BaseModel):
    feature1: float
    feature2: float
    feature3: float

@app.post("/predict")
def predict(data: InputData):
    X = np.array([[data.feature1, data.feature2, data.feature3]])
    prediction = model.predict(X)[0]
    probability = model.predict_proba(X)[0].max()
    return {"prediction": int(prediction), "confidence": float(probability)}

@app.get("/health")
def health():
    return {"status": "ok"}

# Run locally: uvicorn app:app --reload --port 8000

```

B) Containerizing with Docker

```

# Dockerfile
FROM python:3.11-slim

WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt

COPY . .
EXPOSE 8000

CMD ["uvicorn", "app:app", "--host", "0.0.0.0", "--port", "8000"]

# Build & run
docker build -t ml-api .
docker run -p 8000:8000 ml-api

```

C) Deploying to the Cloud (Render / AWS)

- **Render:** connect your GitHub repo, set the build/start command, and it auto-deploys on every push — great free/simple option for portfolio projects.
- **AWS options:** EC2 (full VM control), Elastic Beanstalk (managed deploy), Lambda + API Gateway (serverless, pay-per-request), S3 (store model artifacts/static files), SageMaker (managed ML training & hosting).
- Always log requests, monitor latency/errors, and version your models (e.g. model_v1.pkl, model_v2.pkl) so you can roll back safely.

■ **PROJECT: Final Project: AI Resume Analyzer / Chatbot / Sentiment System / Recommender / Support Assistant**

Pick ONE final project that best matches your target job role and build it end-to-end: data → model → API → deployed app. This is the centerpiece of your portfolio and resume.

Dataset: Depends on chosen project (resumes dataset, chatbot intents, review datasets, etc.)

- Define the problem clearly and pick a suitable model (classic ML vs transformer).
- Build the full pipeline: data cleaning → feature engineering/embeddings → model training → evaluation.
- Wrap the model in a FastAPI service with clear input/output schemas.
- Containerize with Docker and deploy to Render or AWS; get a live public URL.
- Write a clean README with problem statement, architecture diagram, metrics, and demo link — this is what recruiters actually open first.

8.6 Interview Quick-Fire (Week 8)

Q: Why did Transformers replace RNNs/LSTMs for most NLP tasks?

A: Transformers process all tokens in parallel via self-attention instead of sequentially, which is far faster to train and captures long-range dependencies more effectively than RNNs, which suffer from vanishing gradients over long sequences.

Q: What is the difference between BERT and GPT at a high level?

A: BERT is an encoder that reads text bidirectionally and is best for understanding tasks like classification, while GPT is a decoder that generates text autoregressively, one token at a time, left to right.

Q: What is RAG and why is it useful?

A: Retrieval-Augmented Generation retrieves relevant external documents at query time and feeds them into the model's context, letting it answer using current or domain-specific information without the cost of retraining.

Q: Why containerize a model with Docker before deploying?

A: Docker packages the model, code, and all dependencies into a single reproducible image, eliminating 'it works on my machine' issues and making deployment consistent across environments.

WEEK +

Career Readiness: Portfolio, Resume & Interview Prep

Turning 8 weeks of learning into a job offer

9.1 The 70-20-10 Learning Rule

Allocation	Activity	Why
70%	Building projects & solving real problems	This is what actually builds skill and shows up on GitHub/resume
20%	Learning concepts (docs, articles, this guide)	Understanding WHY, not just copy-pasting code
10%	Watching tutorials	Good for orientation only — avoid 'tutorial hell'

■ Tutorial Hell Warning: Watching 100 hours of videos without building anything yourself creates an illusion of competence. Recruiters and interviews test what you can DO, not what you've watched.

9.2 Target Portfolio Checklist

- 5-7 completed ML/DL projects covering regression, classification, clustering, and at least one deep learning project (CV or NLP).
- Every project pushed to GitHub with a clear README (problem, data, approach, results, how to run).
- At least one project fully deployed with a live demo link (Render/AWS/Streamlit).
- A short portfolio website or a pinned GitHub profile README summarizing your skills and projects.
- Kaggle profile with at least 1-2 notebook submissions or a competition attempt.

9.3 Building a Job-Ready Resume

- Lead each project bullet with an action verb + measurable outcome, e.g. "Built a Random Forest loan-approval classifier achieving 91% F1-score, deployed via FastAPI + Docker on Render."
- List tools by category: Languages (Python, SQL), ML/DL (Scikit-learn, XGBoost, TensorFlow/PyTorch), Data (Pandas, NumPy), Deployment (FastAPI, Docker, AWS), Visualization (Power BI, Matplotlib).
- Keep it to 1 page for internships/entry roles — quality of projects beats quantity of buzzwords.
- Tailor keywords to the specific job description (many companies use ATS keyword screening).

9.4 Common Data Science / ML Interview Rounds

Round	What's Tested	How to Prepare
SQL/Python coding	Query writing, data manipulation, DSA basics	LeetCode Easy/Medium SQL & Python, HackerRank
ML case study	Choosing the right model/metric for a business problem	Practice explaining tradeoffs out loud, not just code
ML theory	Bias-variance, overfitting, metric formulas, algorithm intuition	Review this guide's formulas + Interview Quick-Fire sec
Project deep-dive	Your own resume projects — why this model, why this	Be ready to defend every choice you made

Round	What's Tested	How to Prepare
Behavioral/HR	Communication, teamwork, why-this-company	STAR method for past experiences/internships

9.5 Bias-Variance Tradeoff (asked in almost every interview)

$$\text{Total Error} = \text{Bias}^2 + \text{Variance} + \text{Irreducible Error}$$

- **High Bias (Underfitting):** model is too simple, misses patterns — poor performance on both train and test data. Fix: more features, more complex model, less regularization.
- **High Variance (Overfitting):** model is too complex, memorizes training noise — great train performance but poor test performance. Fix: more data, regularization, simpler model, dropout, early stopping, cross-validation.
- The goal is always the sweet spot that minimizes total error on unseen data.

9.6 Essential Platforms

Category	Platforms
Practice	Kaggle, Google Colab
Learning	Krish Naik, CampusX, official documentation
Documentation	Scikit-Learn Docs, TensorFlow Docs, PyTorch Docs, Hugging Face Docs
Coding practice	LeetCode (SQL + Python), HackerRank, StrataScratch (SQL for DS)
Deployment	Render, Streamlit Community Cloud, AWS Free Tier, Hugging Face Spaces

9.7 8-Week Master Checklist

- Week 1 — Python + Math: NumPy/Pandas fluency, linear algebra, stats, gradient descent from scratch. Project: Netflix Data Analysis.
- Week 2 — Preprocessing/EDA: missing values, outliers, encoding, scaling, correlation. Project: Customer Churn Analysis.
- Week 3 — Regression: Linear/Ridge/Lasso, gradient descent, MAE/MSE/RMSE/R2. Project: House Price Prediction.
- Week 4 — Classification: Logistic Regression, KNN, Naive Bayes, confusion matrix, ROC-AUC. Project: Spam Classifier.
- Week 5 — Trees & Ensembles: Decision Tree, Random Forest, AdaBoost, GBM, XGBoost. Project: Loan Approval Prediction.
- Week 6 — Unsupervised: K-Means, Hierarchical, DBSCAN, PCA, t-SNE. Project: Customer Segmentation.
- Week 7 — Deep Learning/CV: ANN, backprop, CNNs, transfer learning (VGG/ResNet/MobileNet). Project: Cats vs Dogs Classifier.
- Week 8 — NLP/Transformers/Deployment: tokenization, embeddings, attention, BERT/GPT, FastAPI, Docker. Final deployed project.

"The goal is not to complete videos. The goal is to build projects, solve problems, and create a portfolio that demonstrates your skills."

All the best

— go build something great. ■